



Full-Stack-Job-Board — The Lesson

Chain C · Project C-1 · the full written lesson, start to finish — compiled from docs/interactive/index.html

1 — The Full-Stack Architecture

Every request makes the same trip: the browser holds **UI state** (what's on screen right now); the API owns **data state** (what's actually true, in Postgres). The API is **stateless** — it keeps nothing in memory between requests. Every request carries its own JWT, so any running instance of the API can answer it, which is exactly what lets C-4 run several copies behind a load balancer.

The trap: it's tempting to reach for a server-side session (an in-memory dict keyed by session id) the first time you need "who's logged in." That works fine with one API process — and breaks the moment you run two. Say C-4 scales this API to 3 replicas behind a load balancer: a session stored in replica #1's memory is invisible to replica #2, so a user whose request happens to land on a different replica gets logged out at random. A JWT sidesteps this entirely because the token itself (not a server-side lookup) proves who the user is — any replica can verify the signature without asking the others.

```
# backend/app/routers/jobs.py – the full trip in one endpoint
@app.get("/jobs", response_model=list[JobRead])
def list_jobs(db: Session = Depends(get_db),
              page: int = 1, size: int = 20):
    offset, limit = paginate(page, size)
    return list(db.scalars(
        select(Job).order_by(Job.created_at.desc())
        .offset(offset).limit(limit)
    ))
```

2 — JWT Authentication

Auth is a four-step pipeline, and `app/security.py` implements exactly these four functions. The password is **never stored** — only its bcrypt hash (`hash_password`). The token is **signed, not encrypted**: anyone who intercepts it can read the payload (`{"sub": "42", "exp": ...}`) in plain text by just base64-decoding it, but only the server — the one holding `SECRET_KEY` — can produce a signature that `decode_token` will accept as valid.

The trap: "signed" gets misread as "encrypted," and that misreading has a real consequence. Say a future feature adds the user's role to the token for a quick client-side check: `payload = {"sub": subject, "role": user.role, "exp": expire}`. That role claim is now sitting in plain text in a token the browser stores and the network carries — anyone who captures that request (a coworker on the same Wi-Fi, a misconfigured logging proxy) can read it with zero cryptography, just `atob()` on the middle segment. The signature stops someone from *forging* a token; it does nothing to stop someone from *reading* one. Anything secret has no business inside a JWT payload.

```
# backend/app/security.py – real code, not a stub
def create_access_token(subject: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        hours=settings.access_token_expire_hours)
    payload = {"sub": subject, "exp": expire}
    return jwt.encode(payload, settings.secret_key, algorithm="HS256")

def decode_token(token: str) -> str | None:
    try:
        return jwt.decode(token, settings.secret_key,
                           algorithms=["HS256"]).get("sub")
    except jwt.JWTError:
        return None
```

3 — React Query: Server State

`useEffect` + `useState` reinvents loading flags, caching, and refetch logic in every single component that needs server data. React Query (TanStack Query) does it once, centrally: `useJobs()` fetches and caches under the key `["jobs"]`; any component calling it shares that one cache entry instead of each firing its own fetch.

The trap: a mutation succeeding does not, by itself, change anything on screen — `useCreateJob`'s `mutationFn` posts the new job and gets a `201` back, but the `["jobs"]` query cache still holds the old list until something tells it to refetch. Say a hiring manager posts a job and the UI doesn't call `invalidateQueries` afterward: the POST genuinely succeeded (it's in the database, a page refresh would show it), but the manager is staring at a job list that hasn't changed and reasonably assumes the submission failed. `onSuccess: () => qc.invalidateQueries({queryKey: ["jobs"]})` is what closes that gap.

```
// frontend/src/hooks/useJobs.ts – real code, not a stub
export function useJobs() {
    return useQuery({ queryKey: ["jobs"],
                      queryFn: () => api.get("/jobs") });
}
export function useCreateJob() {
    const qc = useQueryClient();
    return useMutation({
        mutationFn: (j) => api.post("/jobs", j),
        onSuccess: () => qc.invalidateQueries({ queryKey: ["jobs"] }),
    });
}
```

4 — Database Migrations

The schema changes over time as `app/models.py` grows. Alembic turns each change into a **versioned**, **reversible** file so every environment — your machine, a teammate's, Railway in production — converges to

the same shape by replaying the same steps, instead of each one drifting based on whoever last ran a manual `ALTER TABLE`.

The trap: `alembic revision --autogenerate` diffs your models against the live schema and writes the "obvious" migration — but it compares by name, so a rename looks identical to a drop-and-add. Say a later pass renames `Job.location` to `Job.city`. Autogenerate doesn't see "location became city" — it sees a column named `location` that's gone and an unrelated column named `city` that's new, and writes `drop_column("location") + add_column("city")`. Run that against a database with real rows and every existing job silently loses its location data. The fix is to hand-edit the generated file into `op.alter_column("jobs", "location", new_column_name="city")` before running it — autogenerate is a draft, not an authority.

```
# versions/0002_add_applications.py
def upgrade():
    op.create_table("applications",
        sa.Column("id", sa.Integer, primary_key=True),
        sa.Column("job_id", sa.ForeignKey("jobs.id")),
        sa.Column("user_id", sa.ForeignKey("users.id")))

def downgrade():
    op.drop_table("applications")
```